CLASS ASSIGNMENT – 9
ID : 12341550
DATE : 29-10-2025

Q1:

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 100

sem_t items;
int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
int item_counter = 0;
int items_produced = 0;
int items_consumed = 0;

pthread_mutex_t buffer_mutex;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        int item;
```

```
        pthread_mutex_lock(&buffer_mutex);
        item = produce_item();
        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        items_produced++;
        pthread_mutex_unlock(&buffer_mutex);

        printf("Producer %d produced: %d\n", id, item);
        sem_post(&items);

    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        sem_wait(&items);
        int item;

        pthread_mutex_lock(&buffer_mutex);
        item = buffer[use_ptr];
        use_ptr = (use_ptr + 1) % BUFFER_SIZE;
        items_consumed++;
        pthread_mutex_unlock(&buffer_mutex);

        printf("Consumer %d consumed: %d\n", id, item);

    }
    return NULL;
}

int main() {
    pthread_t prod_threads[2], cons_threads[2];
    int prod_ids[2] = {1, 2};
```

```
    int cons_ids[2] = {1, 2};

    pthread_mutex_init(&buffer_mutex,NULL);
    sem_init(&items, 0, 0);

    printf("Starting Producer-Consumer (BUGGY VERSION)\
n");

    for(int i = 0; i < 2; i++) {
        pthread_create(&prod_threads[i], NULL, producer,
&prod_ids[i]);
        pthread_create(&cons_threads[i], NULL, consumer,
&cons_ids[i]);
    }

    for(int i = 0; i < 2; i++) {
        pthread_join(prod_threads[i], NULL);
        pthread_join(cons_threads[i], NULL);
    }

    sem_destroy(&items);
    pthread_mutex_destroy(&buffer_mutex);

    printf("\n========== FINAL RESULTS
==========\n");
    printf("Total items produced: %d\n", items_produced);
    printf("Total items consumed: %d\n", items_consumed);
    printf("Final fill_ptr: %d\n", fill_ptr);
    printf("Final use_ptr: %d\n", use_ptr);


    return 0;
}
```

output:



explanation:
Issue: Producers and consumers updated shared variables (buffer, fill_ptr, use_ptr, counters) concurrently without locking, so simultaneous writes corrupted the state.

Fix: Added a pthread_mutex_t buffer_mutex and wrapped each critical section (both producer and consumer updates to the shared buffer and counters) with pthread_mutex_lock/pthread_mutex_unlock, keeping the semaphore for availability tracking.

Q2:

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 100

sem_t items;
sem_t empty_slots;
int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
int item_counter = 0;
int items_produced = 0;
int items_consumed = 0;

pthread_mutex_t buffer_mutex;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        int item;
        sem_wait(&empty_slots);
```

```
        pthread_mutex_lock(&buffer_mutex);
        item = produce_item();
        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        items_produced++;
        pthread_mutex_unlock(&buffer_mutex);

        sem_post(&items);
        printf("Producer %d produced: %d\n", id, item);

    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 100000; i++) {
        sem_wait(&items);
        int item;

        pthread_mutex_lock(&buffer_mutex);
        item = buffer[use_ptr];
        use_ptr = (use_ptr + 1) % BUFFER_SIZE;
        items_consumed++;
        pthread_mutex_unlock(&buffer_mutex);

        sem_post(&empty_slots);
        printf("Consumer %d consumed: %d\n", id, item);

    }
    return NULL;
}

int main() {
    pthread_t prod_threads[2], cons_threads[2];
```

```
    int prod_ids[2] = {1, 2};
    int cons_ids[2] = {1, 2};

    pthread_mutex_init(&buffer_mutex,NULL);
    sem_init(&items, 0, 0);
    sem_init(&empty_slots,0,BUFFER_SIZE);

    printf("Starting Producer-Consumer (BUGGY VERSION)\
n");

    for(int i = 0; i < 2; i++) {
        pthread_create(&prod_threads[i], NULL, producer,
&prod_ids[i]);
        pthread_create(&cons_threads[i], NULL, consumer,
&cons_ids[i]);
    }

    for(int i = 0; i < 2; i++) {
        pthread_join(prod_threads[i], NULL);
        pthread_join(cons_threads[i], NULL);
    }

    sem_destroy(&items);
    sem_destroy(&empty_slots);
    pthread_mutex_destroy(&buffer_mutex);

    printf("\n========== FINAL RESULTS
==========\n");
    printf("Total items produced: %d\n", items_produced);
    printf("Total items consumed: %d\n", items_consumed);
    printf("Final fill_ptr: %d\n", fill_ptr);
    printf("Final use_ptr: %d\n", use_ptr);


    return 0;
```

```
}
```

output:



explanation:

The buffer is now bounded: empty_slots tracks free cells and forces producers to wait when the array is full, while items keeps consumers waiting until work exists. Both producer and consumer mutate the shared indices under buffer_mutex, so buffer state stays consistent even with multiple threads.

Q3:

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

sem_t full_slots;
sem_t empty_slots;

pthread_mutex_t mutex;
int buffer[BUFFER_SIZE];
int fill_ptr = 0;
int use_ptr = 0;
int item_counter = 0;

int produce_item() {
    return item_counter++;
}

void* producer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 10; i++) {
        int item = produce_item();

        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);
```

```
        buffer[fill_ptr] = item;
        fill_ptr = (fill_ptr + 1) % BUFFER_SIZE;
        printf("Producer %d produced: %d\n", id, item);

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);
        usleep(100000);
    }
    return NULL;
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 10; i++) {
        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);

        int item = buffer[use_ptr];
        use_ptr = (use_ptr + 1) % BUFFER_SIZE;
        printf("Consumer %d consumed: %d\n", id, item);

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);
        usleep(150000);
    }
    return NULL;
}

int main() {
    pthread_t prod1, cons1;
    int prod_id = 1, cons_id = 1;

    sem_init(&full_slots, 0, 0);
    sem_init(&empty_slots, 0, BUFFER_SIZE);
```

```
    pthread_mutex_init(&mutex, NULL);

    printf("Starting code - WILL DEADLOCK!\n");

    pthread_create(&prod1, NULL, producer, &prod_id);
    pthread_create(&cons1, NULL, consumer, &cons_id);

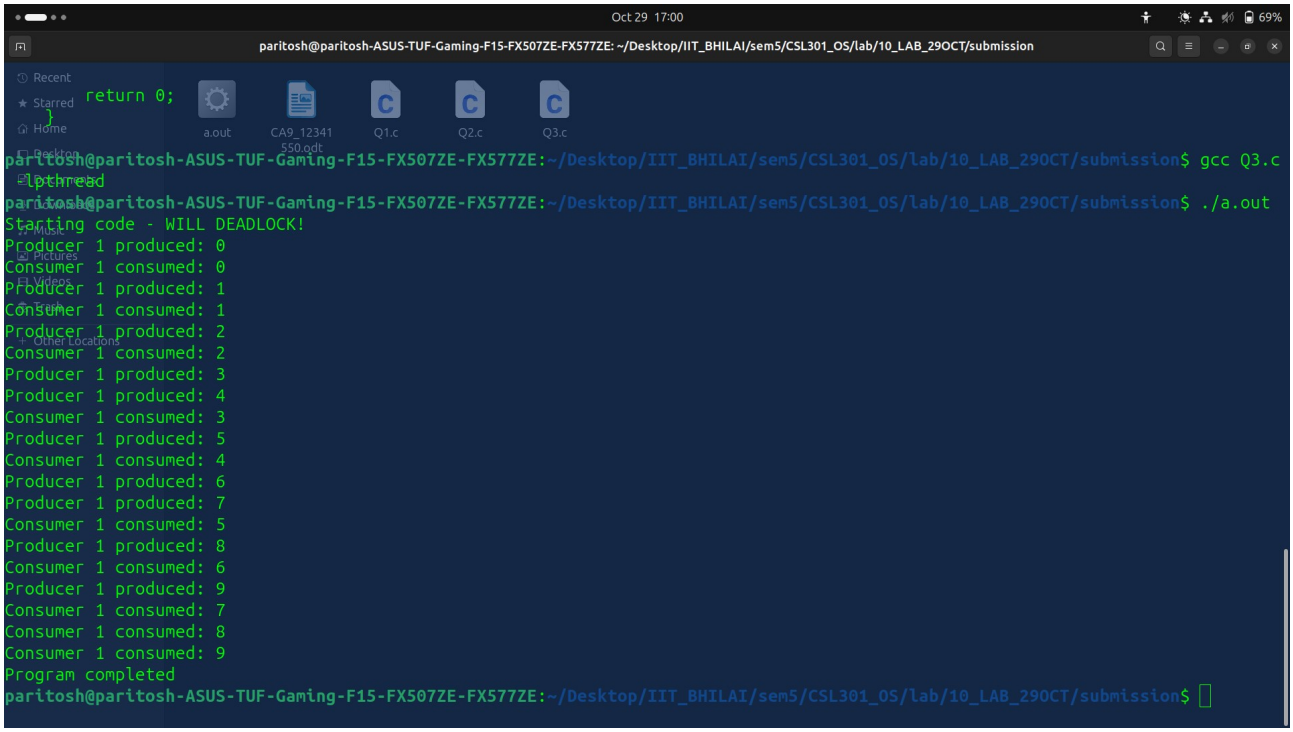
    pthread_join(prod1, NULL);
    pthread_join(cons1, NULL);

    sem_destroy(&full_slots);
    sem_destroy(&empty_slots);

    pthread_mutex_destroy(&mutex);
    printf("Program completed\n");

    return 0;
}
```

Output:



explanation:
Original issue: both threads locked the mutex first and then blocked on sem_wait, so each held mutex while waiting for the other to post deadlock.
Fix: make producers wait on empty_slots and consumers wait on full_slots before taking the mutex, update the buffer while holding the mutex, then release it and signal the opposite semaphore.

Q4:

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t roomEmpty;

pthread_mutex_t reader_mutex;

int readers = 0;
int shared_data = 0;
int read_count = 0;
int write_count = 0;

void* reader(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 50; i++) {

        pthread_mutex_lock(&reader_mutex);
        readers++;
        if(readers == 1) {
```

```
            sem_wait(&roomEmpty);
        }
        pthread_mutex_unlock(&reader_mutex);

        int value = shared_data;
        read_count++;
        printf("Reader %d reads: %d (thinks readers=%d)\n", id,
value, readers);

        pthread_mutex_lock(&reader_mutex);
        readers--;
        if(readers == 0) {
            sem_post(&roomEmpty);
        }
        pthread_mutex_unlock(&reader_mutex);

        usleep(10000);
    }
    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;
    for(int i = 0; i < 20; i++) {
        sem_wait(&roomEmpty);

        // Writing
        shared_data++;
        write_count++;
        printf("Writer %d writes: %d\n", id, shared_data);
        usleep(50000);

        sem_post(&roomEmpty);
        usleep(100000);
    }
```

```
    return NULL;
}

int main() {
    pthread_t reader_threads[4], writer_threads[2];
    int reader_ids[4] = {1, 2, 3, 4};
    int writer_ids[2] = {1, 2};

    sem_init(&roomEmpty, 0, 1);
    pthread_mutex_init(&reader_mutex,NULL);

    printf("=== BUGGY VERSION - Watch for anomalies ===\
n");
    printf("Expected behavior: Writers should NEVER run while
readers>0\n\n");

    for(int i = 0; i < 4; i++) {
        pthread_create(&reader_threads[i], NULL, reader,
&reader_ids[i]);
    }
    for(int i = 0; i < 2; i++) {
        pthread_create(&writer_threads[i], NULL, writer,
&writer_ids[i]);
    }

    for(int i = 0; i < 4; i++) {
        pthread_join(reader_threads[i], NULL);
    }
    for(int i = 0; i < 2; i++) {
        pthread_join(writer_threads[i], NULL);
    }

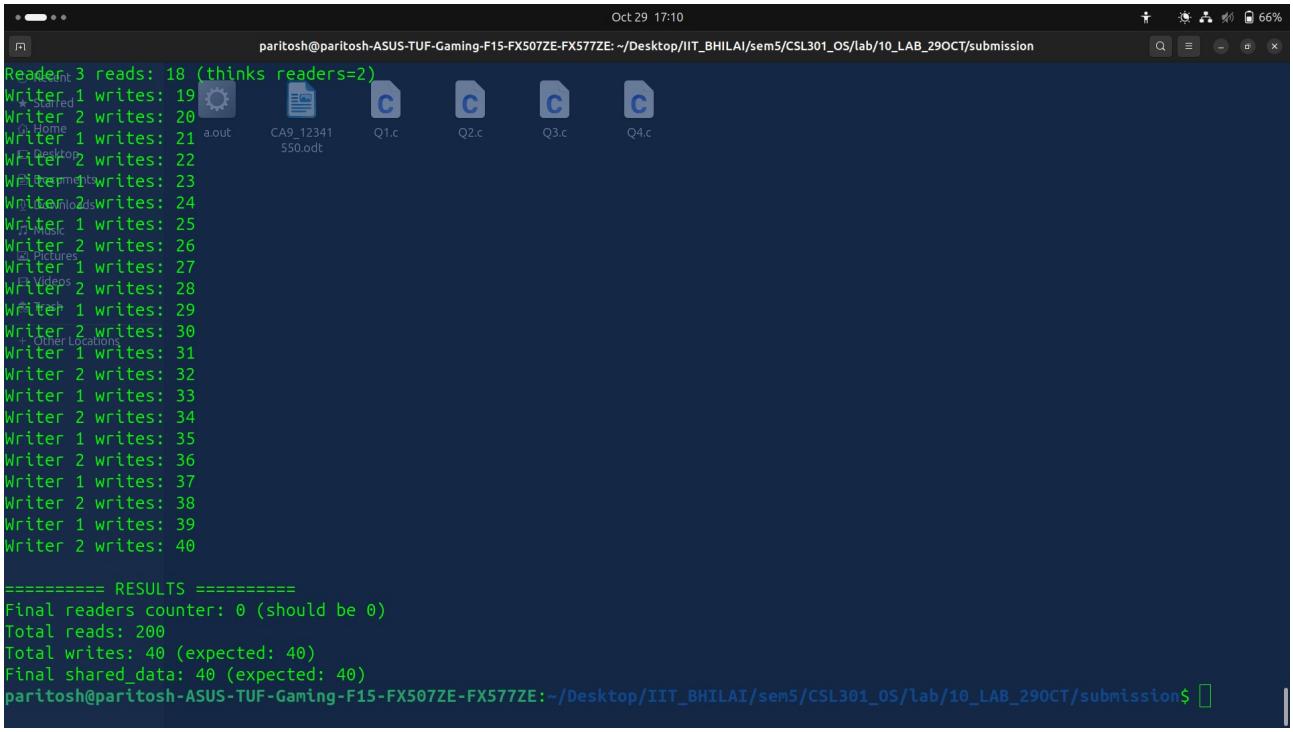
    sem_destroy(&roomEmpty);
    pthread_mutex_destroy(&reader_mutex);
```

```
        printf("\n========== RESULTS ==========\n");
        printf("Final readers counter: %d (should be 0)\n", readers);
        printf("Total reads: %d\n", read_count);
        printf("Total writes: %d (expected: 40)\n", write_count);
        printf("Final shared_data: %d (expected: 40)\n",
shared_data);

        if(readers != 0) {
            printf("\nBUG DETECTED: readers counter is
corrupted!\n");
        }
        if(write_count != 40 || shared_data != 40) {
            printf("BUG DETECTED: Data corruption occurred!\n");
        }

        return 0;
    }
```

output:



explanation:
Problem: readers updated the shared readers counter without synchronization, so races let writers slip in and corrupt shared_data.
Fix: guard increments/decrements of readers with a new reader_mutex; first reader grabs roomEmpty, last reader releases it, preventing writers from entering while readers are active.

Q5:

code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>


typedef struct {
    int counter;
    sem_t mutex;
} Lightswitch;

Lightswitch readSwitch;
sem_t roomEmpty;


void lightswitch_init(Lightswitch *ls) {
    ls->counter = 0;
    sem_init(&ls->mutex, 0, 1);
}


void lightswitch_lock(Lightswitch *ls, sem_t *semaphore) {
    sem_wait(&ls->mutex);
```

```
    ls->counter++;
    if (ls->counter == 1) {
        sem_wait(semaphore);
    }
    sem_post(&ls->mutex);
}


void lightswitch_unlock(Lightswitch *ls, sem_t *semaphore) {
    sem_wait(&ls->mutex);
    ls->counter--;
    if (ls->counter == 0) {
        sem_post(semaphore);
    }
    sem_post(&ls->mutex);
}

// ----- Complete Reader Thread -----
void *reader(void *arg) {
    int id = *(int *)arg;

    lightswitch_lock(&readSwitch, &roomEmpty);
    printf("Reader %d is reading...\n", id);
    sleep(1);
    printf("Reader %d finished reading.\n", id);
    lightswitch_unlock(&readSwitch, &roomEmpty);

    return NULL;
}

// ----- Complete Writer Thread -----
void *writer(void *arg) {
    int id = *(int *)arg;

    sem_wait(&roomEmpty);
```

```
    printf("Writer %d is writing...\n", id);
    sleep(2);
    printf("Writer %d finished writing.\n", id);

    sem_post(&roomEmpty);

    return NULL;
}

int main() {
    pthread_t r1, r2, w1, w2;
    int rID1 = 1, rID2 = 2;
    int wID1 = 1, wID2 = 2;


    lightswitch_init(&readSwitch);
    sem_init(&roomEmpty, 0, 1);

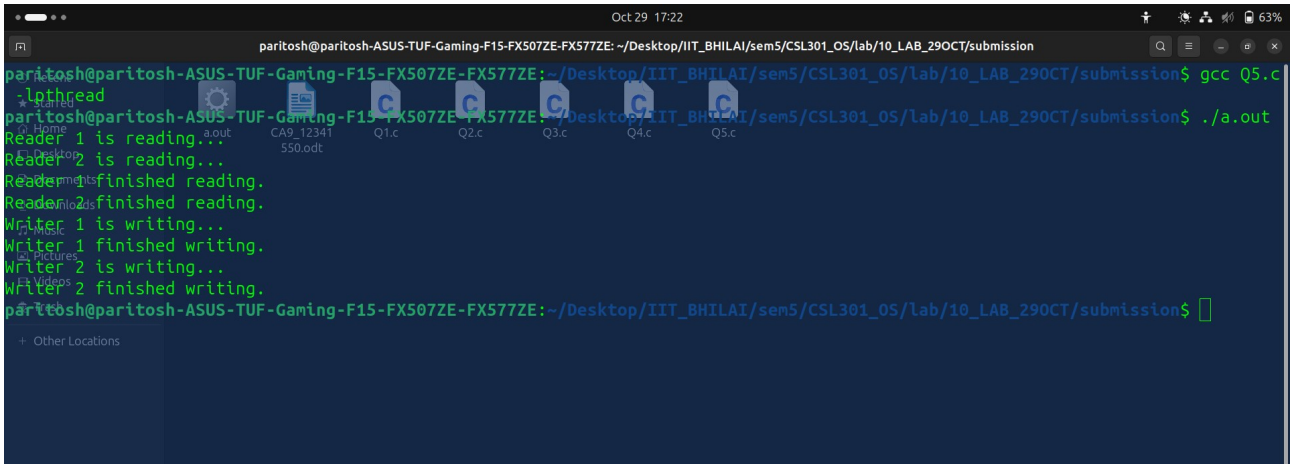

    pthread_create(&r1, NULL, reader, &rID1);
    pthread_create(&r2, NULL, reader, &rID2);
    pthread_create(&w1, NULL, writer, &wID1);
    pthread_create(&w2, NULL, writer, &wID2);


    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(w1, NULL);
    pthread_join(w2, NULL);


    sem_destroy(&roomEmpty);
    sem_destroy(&readSwitch.mutex);
```

```
    return 0;
}
```

output:



explanation:
Readers share access via the lightswitch: the first reader acquires roomEmpty, additional readers bypass writers, and the last reader releases it. Writers simply wait on roomEmpty, guaranteeing exclusive access.

Q6:

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define N 5

sem_t forks[N];
int eat_count[N] = {0};

void* philosopher(void* arg) {
    int id = *(int*)arg;
    int left_fork = id;
    int right_fork = (id + 1) % N;
    int first_fork = left_fork < right_fork ? left_fork : right_fork;
    int second_fork = left_fork < right_fork ? right_fork :
left_fork;

    for(int i = 0; i < 3; i++) {

        printf("Philosopher %d is thinking...\n", id);
        usleep(100000);


        printf("Philosopher %d is hungry, reaching for forks %d
and %d\n",
            id, first_fork, second_fork);

        sem_wait(&forks[first_fork]);
        printf("  Philosopher %d picked up left fork %d\n", id,
first_fork);
        usleep(50000);

        sem_wait(&forks[second_fork]);
        printf("  Philosopher %d picked up right fork %d\n", id,
second_fork);


        printf("Philosopher %d is EATING (meal #%d)\n", id,
eat_count[id] + 1);
```

```
            eat_count[id]++;
            usleep(200000);


            sem_post(&forks[first_fork]);
            sem_post(&forks[second_fork]);
            printf("  Philosopher %d put down both forks\n\n", id);
        }

        return NULL;
    }

    int main() {
        pthread_t philosophers[N];
        int ids[N];


        for(int i = 0; i < N; i++) {
            sem_init(&forks[i], 0, 1);
            ids[i] = i;
        }

        printf("=== DINING PHILOSOPHERS - DEADLOCK
VERSION ===\n");
        printf("Number of philosophers: %d\n", N);
        printf("Watch for deadlock...\n\n");


        for(int i = 0; i < N; i++) {
            pthread_create(&philosophers[i], NULL, philosopher,
&ids[i]);
        }


        for(int i = 0; i < N; i++) {
```

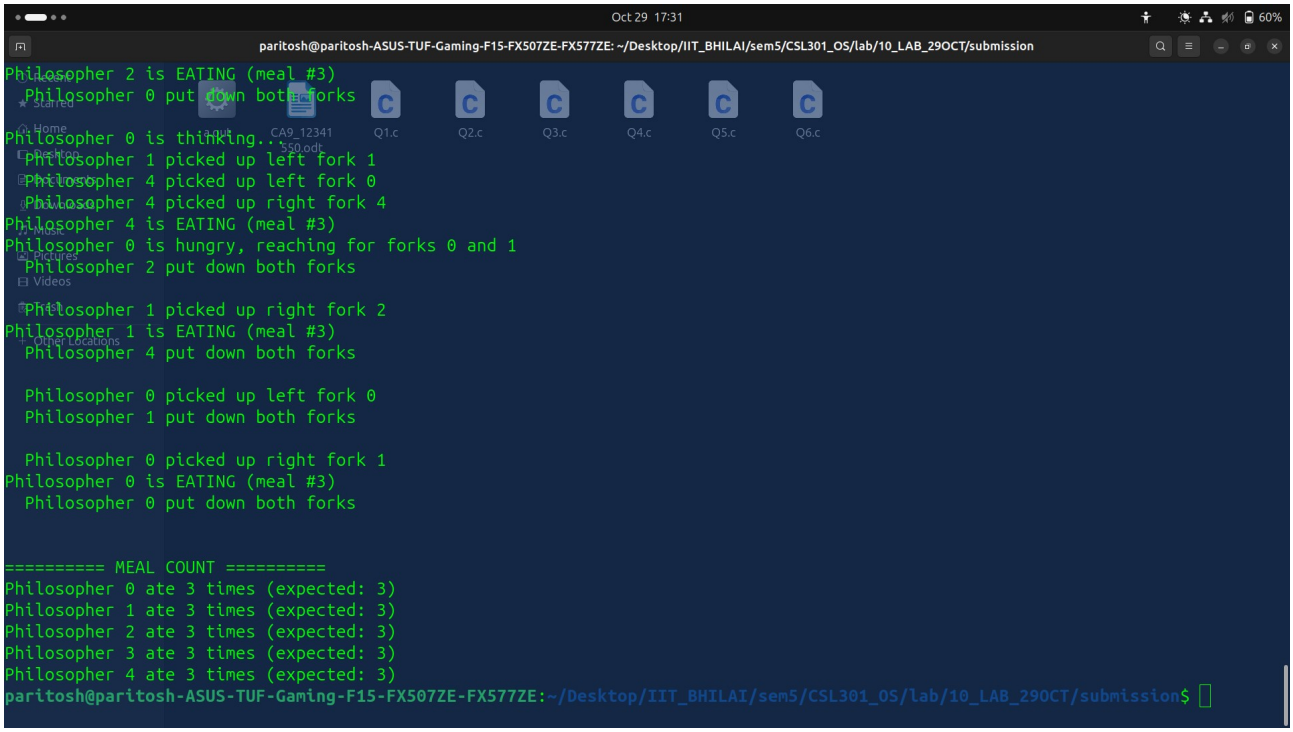
```
        pthread_join(philosophers[i], NULL);
    }


    for(int i = 0; i < N; i++) {
        sem_destroy(&forks[i]);
    }

    printf("\n========== MEAL COUNT ==========\
n");
    for(int i = 0; i < N; i++) {
        printf("Philosopher %d ate %d times (expected: 3)\n", i,
eat_count[i]);
    }

    return 0;
}
```

output:



explanation:
Problem: all philosophers originally grabbed their left fork first, so every thread could hold one fork while waiting for the other, producing deadlock.
Fix: each philosopher now locks the lower-numbered fork first and the higher-numbered fork second, ensuring at least one thread can always complete the pair and eat.